Innovation Centre for Education

YENEPOYA

(DEEMED TO BE UNIVERSITY)

Project –High Level Design

on

AI-Driven Self-Healing Cloud for Healthcare.

Student Name: ASHIYANA HIBA

Industry Mentor: **Guided by:**

Table of Contents

1. Introduction. 6
   1.1. Scope of the document. 6
   1.2. Intended Audience 6
   1.3. System overview 6

2. System Design. 6
   2.1. Application Design 6
   2.2. Process Flow. 6
   2.3. Information Flow. 7
   2.4. Components Design 8
   2.5. Key Design Considerations 8
   2.6. API Catalogue. 8

3. Data Design. 8
   3.1. Data Model 8
   3.2. Data Access Mechanism 9
   3.3. Data Retention Policies 9
   3.4. Data Migration 9

4. Interfaces 9

5. State and Session Management 10

6. Caching 10

7. Non-Functional Requirements 10
   7.1. Security Aspects 10
   7.2. Performance Aspects 10

8. References 11

1. Introduction

1.1 Scope of the document

This document presents the design, development, and implementation of the project titled **"AIDriven Self-Healing Cloud for Healthcare."** It provides a comprehensive overview of the system architecture, functional components, process flow, and technical implementation.

The scope of this document includes:

- Understanding self-healing cloud concepts
- Designing a monitoring and recovery system
- Implementing automated process monitoring using Python
- Visualizing system behavior through a graphical dashboard

The document serves as a complete guide for understanding how automation can be applied in healthcare systems to ensure reliability and continuous service availability.

1.2 Intended Audience

This document is intended for:

- **Students** pursuing BCA and related programs
- **Faculty members** evaluating the project
- **Developers** interested in DevOps and automation
- **Beginners** learning cloud computing concepts
- **Researchers** exploring self-healing systems in healthcare

1.3 System Overview

The **AI-Driven Self-Healing Cloud for Healthcare** system is designed to simulate real-world cloud-based healthcare service monitoring and recovery mechanisms.

Healthcare systems require high availability, and any downtime can lead to serious consequences. This project addresses that issue by implementing a system that:

- Continuously monitors critical services
- Detects failures in real time
- Automatically restarts failed processes

- Displays system status through a dashboard

The system uses Python for backend logic and Streamlit for visualization. It mimics how largescale cloud platforms maintain system reliability using automated self-healing techniques.

2. System Design

2.1 Application Design

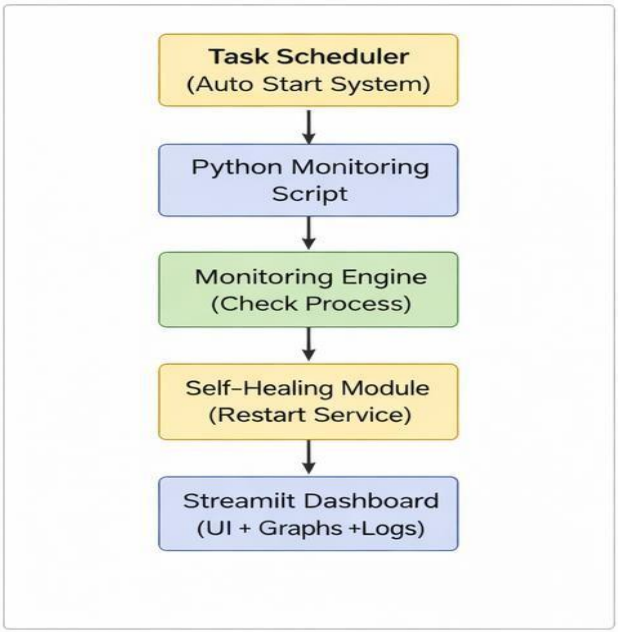


Figure 2.1: Application Design Diagram

The application follows a modular architecture where different components handle monitoring, decision-making, recovery, and visualization.

The core idea is to create a loop-based monitoring system that checks the status of a target process (e.g., Notepad). If the process is not running, the system triggers a restart mechanism.

The application consists of:

- Backend monitorg engine
- Frontend dashboard (Streamlit)
- Scheduler integration (Task Scheduler)

This design ensures simplicity while demonstrating real-world cloud automation concepts.

2.2 Process Flow

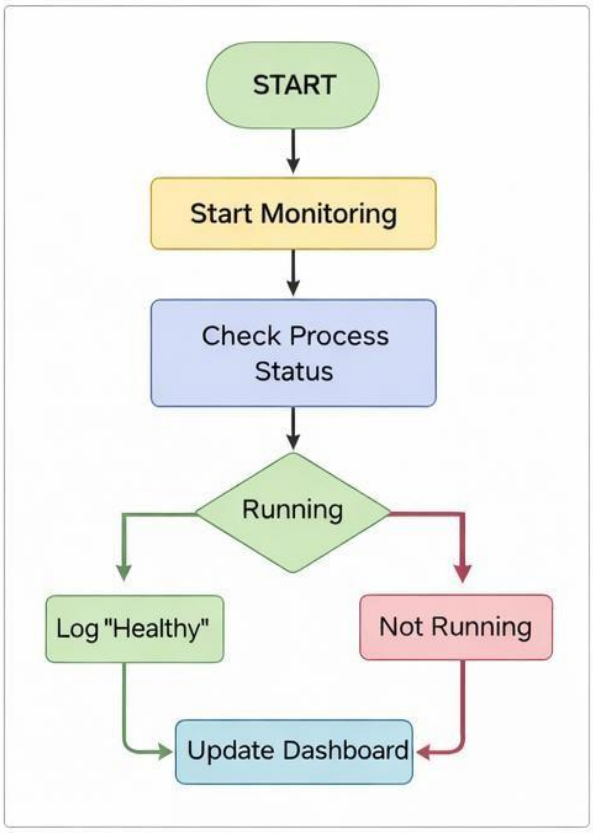


Figure 2.2: Process Flow Diagram

1. The system starts via manual execution or Task Scheduler
2. The monitoring script begins execution
3. It checks whether the target process is running
4. If running → system logs “healthy state”
5. If not running → system detects failure
6. Restart command is executed automatically
7. Dashboard updates system status and logs
8. The loop continues at fixed intervals

2.3 Information Flow

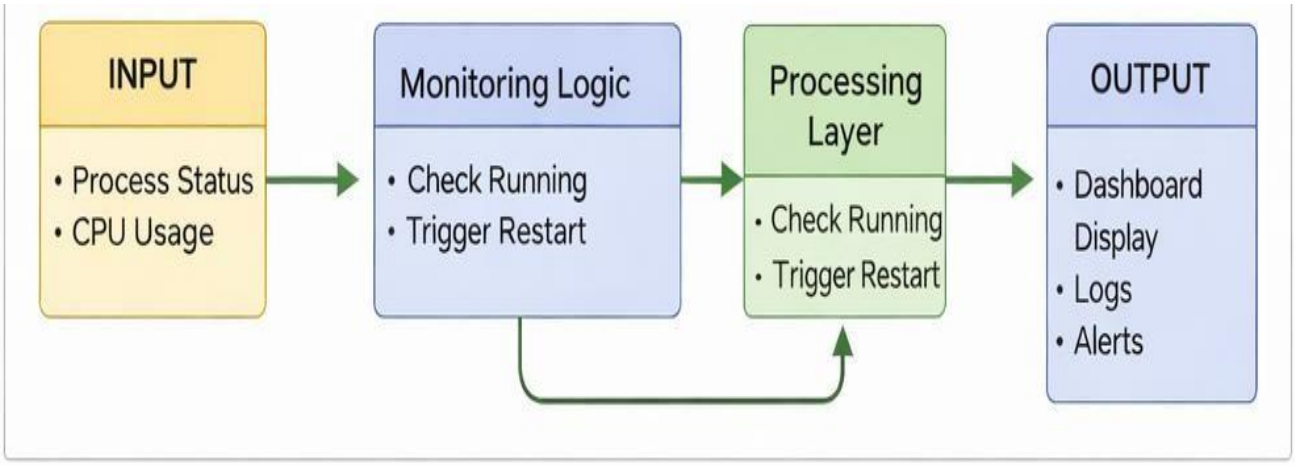


Figure 2.3: Information Flow Diagram

Input:

- Process list from OS
- CPU and memory usage data

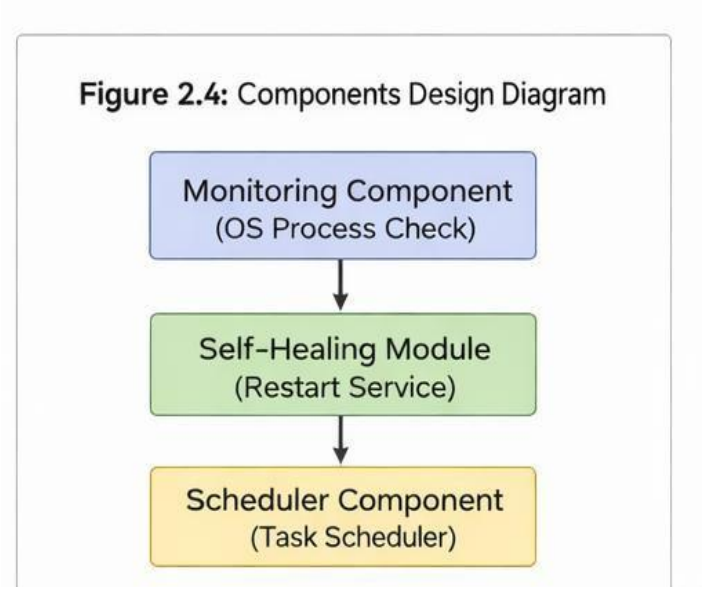
Processing:

- Process verification logic
- Decision-making (running or not)
- Triggering restart if needed

Output:

- Dashboard display
- Logs of system activity
- Alerts for failures

2.4 Components Design



1. Monitoring Component

Responsible for checking system processes using OS-level commands.

2. Self-Healing Component

Triggers automatic restart of services when failure is detected.

3. Visualization Component

Displays system metrics, logs, and alerts using Streamlit.

4. Scheduler Component

Automates execution using Task Scheduler to simulate real cloud environments.

2.5 Key Design Considerations

- **Reliability:** Ensures continuous system monitoring
- **Automation:** Reduces manual intervention

Scalability: Can be extended to multiple services
Performance: Lightweight and efficient
Usability: Simple and interactive interface

2.6 API Catalogue

API Name	Description	Input	Output
is_running()	Checks process status	Process name	Boolean
restart_service()	Restarts service	Process name	Status
get_cpu_usage()	Retrieves CPU usage	None	Percentage
log_event()	Logs system activity	Message	Log entry
update_dashboard()	Updates UI	Data	Visual output

3. Data Design

3.1 Data Model

The system uses a simple data model consisting of:

- Process status data
- CPU usage metrics
- Log entries

These are stored temporarily in memory during execution.

3.2 Data Access Mechanism

Data is accessed using:
Python OS library for process information
psutil library for system metrics
Streamlit session state for storing logs

3.3 Data Retention Policies

- Logs are stored temporarily during runtime
- Only recent logs (last 10–20 entries) are displayed
- No long-term storage is implemented

3.4 Data Migration

Since the system does not use persistent databases, data migration is not required. Future versions can integrate cloud storage for log retention.

4. Interfaces

The system provides a graphical user interface using Streamlit.

Features include:

- Dashboard displaying system status
- CPU usage graphs
- Log display panel
- Start/Stop monitoring controls

The interface is designed to be user-friendly and visually appealing.

5. State and Session Management

The system uses Streamlit session state to manage:

- Monitoring status (running/stopped)
- Log data
- CPU usage history

This ensures continuity during execution and prevents data loss during UI refresh.

6. Caching

Caching is implemented using Streamlit mechanisms to:

- Store recent CPU usage values
- Avoid redundant computations
- Improve performance

7. Non-Functional Requirements

7.1 Security Aspects

- No sensitive data is processed
- Local system execution ensures safety
- Can be extended with authentication in future

7.2 Performance Aspects

- Lightweight Python script
- Minimal CPU usage
- Real-time monitoring with low latency
- Efficient loop execution

8. References

1. Python Official Documentation – https://docs.python.org
2. Streamlit Documentation – https://docs.streamlit.io
3. psutil Library Documentation – https://psutil.readthedocs.io
4. Microsoft Task Scheduler Guide
5. Cloud Computing Concepts – Self-Healing Systems